

5-5 · 파티셔닝과 셔플링 최적화 · 실습 1.5H

repartition/coalesce와 셔플 관찰

파티션 수를 바꿔 가며 시간과 셔플 지표를 직접 쟀다

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1.5시간 뒤에는

- ① 파티션 수 변화를 `getNumPartitions()` 로 확인한다
- ② Spark UI 에서 Stage 수와 Shuffle Read/Write 를 읽는다
- ③ `shuffle.partitions` 를 200·32·8 로 바꿔 시간을 비교한다
- ④ 스큐를 만들어 한 태스크만 오래 걸리는 것을 본다

젠 값(파티션 수·실행 시간·파일 수)은 다음 1H 산출물 실측표에 그대로 적습니다

여섯 단계 — PySpark 셀 하나에서 끝까지

5-4 에서 쓴 옵션 그대로 PySpark 셀을 열고 시작합니다

1. 파티션 수	2. 분포	3. 셔플 관찰	4. 셔플 파티션	5.6. 스큐·파일
repartition(8) coalesce(2)	파티션별 행 수	UI Stages explain()	200·32·8 시간 비교	hot 키 coalesce(1)

핵심 명령 — 파티션 수와 분포

1천만 행	<code>df = spark.range(1, 10_000_001)</code>
파티션 수	<code>df.rdd.getNumPartitions()</code> # 환경에 따라 다름
다시 나누기	<code>df.repartition(8)</code> → 8
합치기	<code>repartitioned.coalesce(2)</code> → 2
파티션별 행 수	<code>groupBy(F.spark_partition_id()).count()</code>

`coalesce(16)` 처럼 현재보다 큰 값을 주면 파티션이 늘지 않고 그대로 — 이론에서 말한 그대로

셔플을 Spark UI 와 explain() 에서 관찰

1

(A) 좁은 변환만 — filter 후 count

Stages 탭에 Stage 1개 · explain() 에 Exchange 없음

2

(B) 넓은 변환 — groupBy(bucket).count

Stage 2개 · 앞 Stage 에 Shuffle Write, 뒤 Stage 에 Shuffle Read

3

explain() 만으로도 판정

두 번째에만 Exchange hashpartitioning — UI 를 안 열어도 셔플 여부를 안다

애플리케이션 UI

4040

Stages 탭

셔플 파티션 200·32·8 로 재 보기

shuffle.partitions = 200 (기본)

그룹이 10개뿐인데 200조각 — 빈 태스크 190개

- bench(200) 이 가장 느린 경향
- Task 수 200
- AQE 가 켜져 있으면 차이가 작을 수 있다

VS

shuffle.partitions = 8

그룹 수에 가까운 조각 — 빈 태스크가 거의 없다

- bench(8) 이 빠른 경향
- 재고 나서 200 으로 원복
- adaptive.enabled 가 true 면 자동 조정

스큐 — 90% 가 한 키에 몰리면

- 1 hot 키에 90% 를 몰아넣는다**
F.when(id % 10 < 9, "hot") — 고른 분포(id % 10)와 짝으로
- 2 groupBy(k).count() 시간을 쟀다**
스큐 있는 쪽이 느리다 — 평균은 괜찮은데 전체가 느린 전형
- 3 UI 에서 Task Duration 을 본다**
한 태스크만 유난히 오래 — 전체는 그 태스크를 기다린다

Max ÷ Median 이

**5배 넘으면
스큐 의심**

해결책은 키에 랜덤 접미사를 붙여 흩뿌리는 솔팅(salting) — 이번 과정 범위 밖, 개념만

저장 파일 수를 coalesce 로 조절

집계	<code>agg = df.withColumn("b", F.col("id") % 10).groupBy("b").count()</code>
그냥 저장	<code>agg.write.mode("overwrite").parquet("s3a://warehouse/test-many/")</code>
정리해서 저장	<code>agg.coalesce(1).write.mode("overwrite").parquet("../test-one/")</code>
파일 세기	<code>docker run --rm --network week5_default minio/mc sh -c ...</code>
개수	<code>mc ls m/warehouse/test-many/ grep -c parquet</code>

test-many 는 여러 개, test-one 은 1개 — 4주차의 rewrite_data_files 가 이 문제를 뒤늦게 고치던 도구

이 1.5시간이 끝날 때 확인할 것

- 1 repartition(8) 후 8, coalesce(2) 후 2, coalesce(16) 후에도 2
- 2 repartition 은 고르게, coalesce 는 치우친 분포
- 3 groupBy 는 Stage 2개 · Shuffle Write/Read 가 기록된다
- 4 shuffle.partitions 가 작을수록 빠른 경향(AQE 면 차이 작음)
- 5 스큐 쪽이 느리고 Task Max 가 Median 보다 훨씬 크다
- 6 test-one 의 parquet 파일은 1개

5-5 · 실습 1.5H 마무리

다음은 배치 스크립트로 데이터 마트 (1H)

지금까지의 가공을 mart.py 한 파일로 묶어 spark-submit 만으로 돌게 만듭니다 — 그것이 5주차 산출물입니다

✓ mart.py 작성 · spark-submit 실행

✓ 두 번 돌려 먹등 확인

✓ 날짜 인자·로그·종료 코드 추가

✓ 실측표를 채워 산출물 정리

가져갈 것

파티션 수 · 200 vs 8 시간 · 파일 수